

FPGA-like LUT based Configurable Logic Block

Contents:

1. Abstract
2. Introduction
3. Block diagram
4. Objectives
5. Theory of Operation
6. Schematics
7. Simulations Results & Observations
8. Applications
9. Conclusion

1.ABSTRACT:

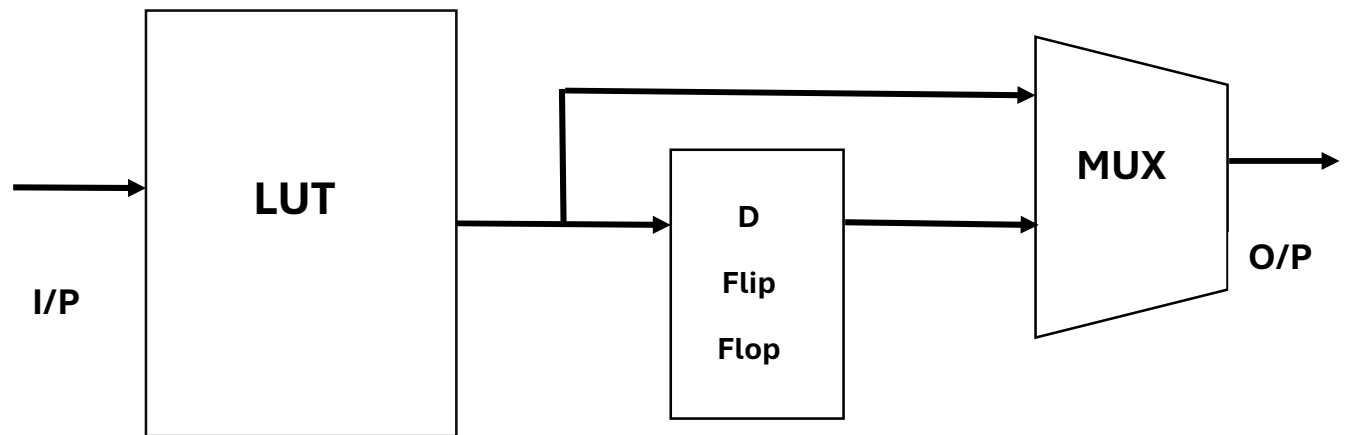
This work presents the design and simulation of an FPGA-like configurable logic block (CLB) implemented at the transistor level in LTspice. The proposed architecture is based on a 4-input Lookup Table (LUT) that stores a 16-bit truth table using a Serial-In Parallel-Out (SIPO) shift register and sixteen static SRAM cells. A 4:16 address decoder selects one of the sixteen stored configuration bits depending on the applied input vector, and the selected value is optionally synchronized using a D-flip-flop before being passed to the output multiplexer. This structure emulates the fundamental logic fabric used in commercial FPGA devices, where logic functions are realized by programming memory cells that drive multiplexers. The design enables implementation of any arbitrary 4-input logic function by simply reloading the 16-bit LUT configuration. All blocks—SIPO register, SRAM storage array, decoder, D-FF, and MUX—were modeled using CMOS transistor-level circuits and validated through transient simulation. The proposed CLB demonstrates reconfigurability for FPGA architecture.

2.INTRODUCTION

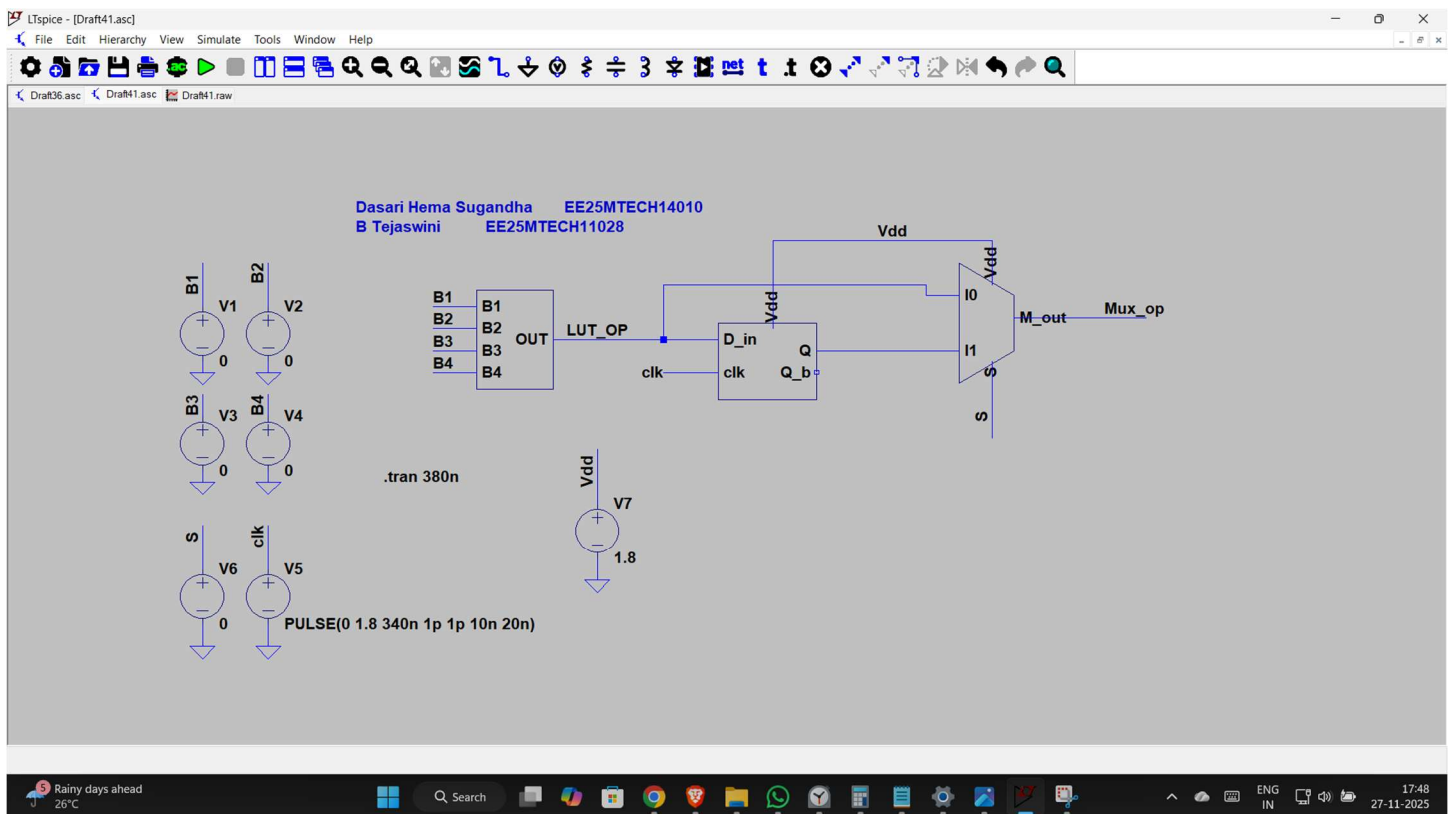
Field-Programmable Gate Arrays (FPGAs) rely on a massively programmable array of logic blocks, where each logic block can be configured to implement different Boolean functions. The heart of every FPGA logic block is the **Lookup Table (LUT)**, a compact memory that stores the truth table of the intended logic function. By selecting one entry of this memory based on input signals, the LUT outputs the corresponding result, enabling the FPGA to perform combinational computation without requiring fixed logic gates.

In this project, we design a **4-input LUT-based Configurable Logic Block (CLB)** that closely resembles a real FPGA logic element. The architecture uses a **16-bit Serial-In Parallel-Out (SIPO) shift register** to load function data serially, making the LUT programmable. The function bits are stored in **sixteen 6T SRAM cells**, which act as memory elements of the LUT. A **4:16 address decoder** selects one of the sixteen SRAM bits based on the external 4-bit input vector. To provide optional synchronous behavior, the output of the LUT is latched using a **D flip-flop** before being fed to the final multiplexer, which produces the CLB output.

3. BLOCK DIAGRAM(theoretical)



Block Diagram(in LTspice)



4. OBJECTIVES:

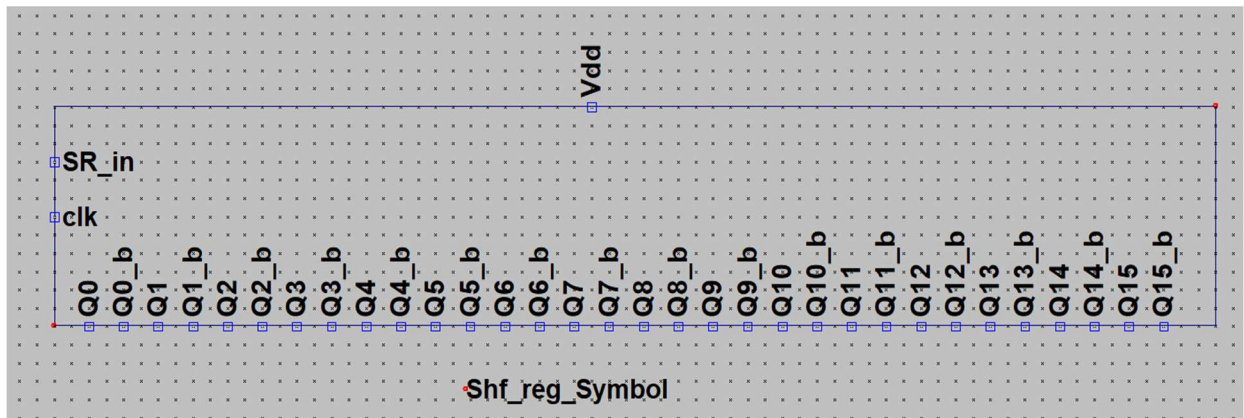
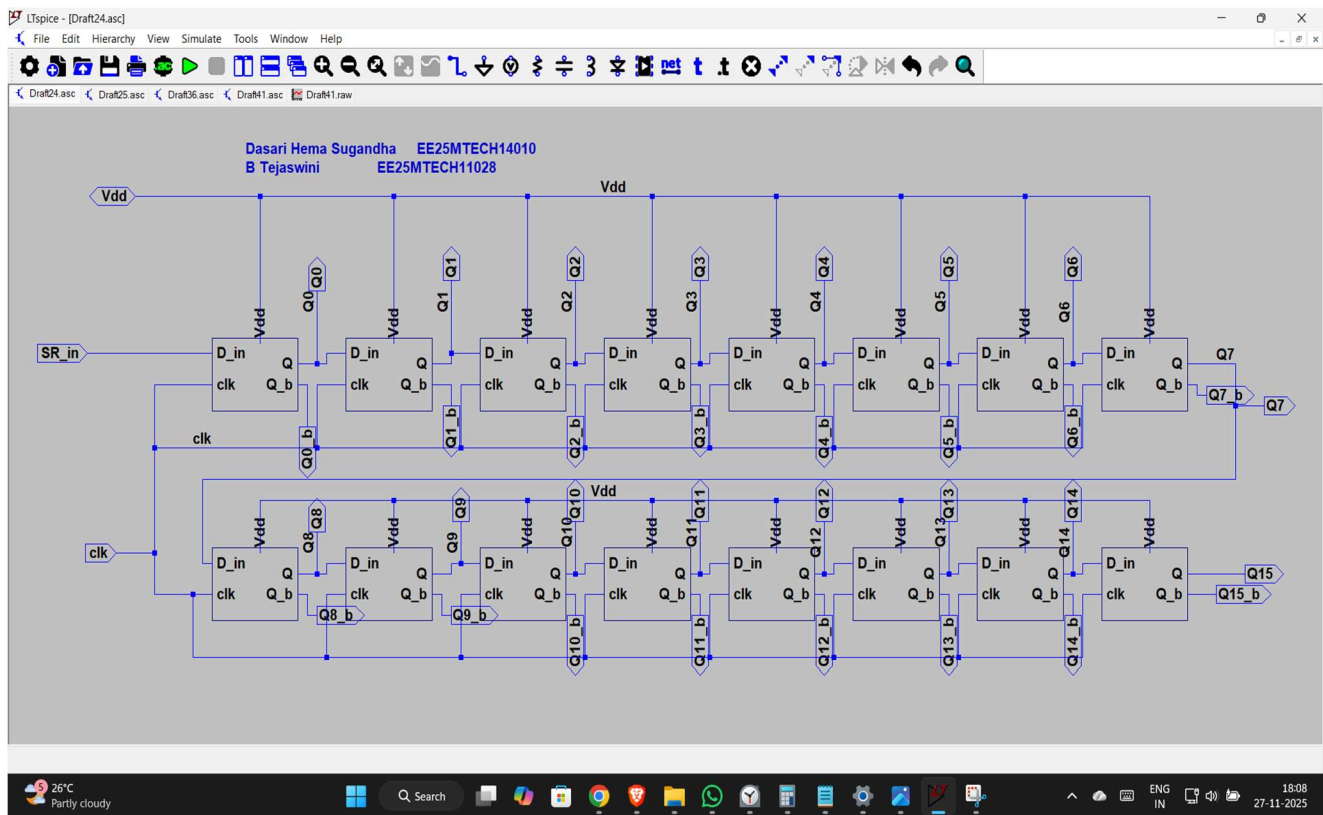
1. To implement a reconfigurable logic element using a transistor-level LUT architecture.
2. There's no separate circuitry required for any 4-bit input and 16-bit output of any Boolean function.
3. To design a 16-bit Serial-In Parallel-Out (SIPO) shift register for LUT programming.
4. To store LUT configuration using sixteen CMOS SRAM cells.
5. To design a 4:16 decoder for LUT addressing.
6. To implement a MUX-based readout structure resembling real FPGA LUTs.
7. To validate the complete CLB via LTspice simulation.

5.THEORY OF OPERATION:

1)SIPO(Serial in Parallel out):

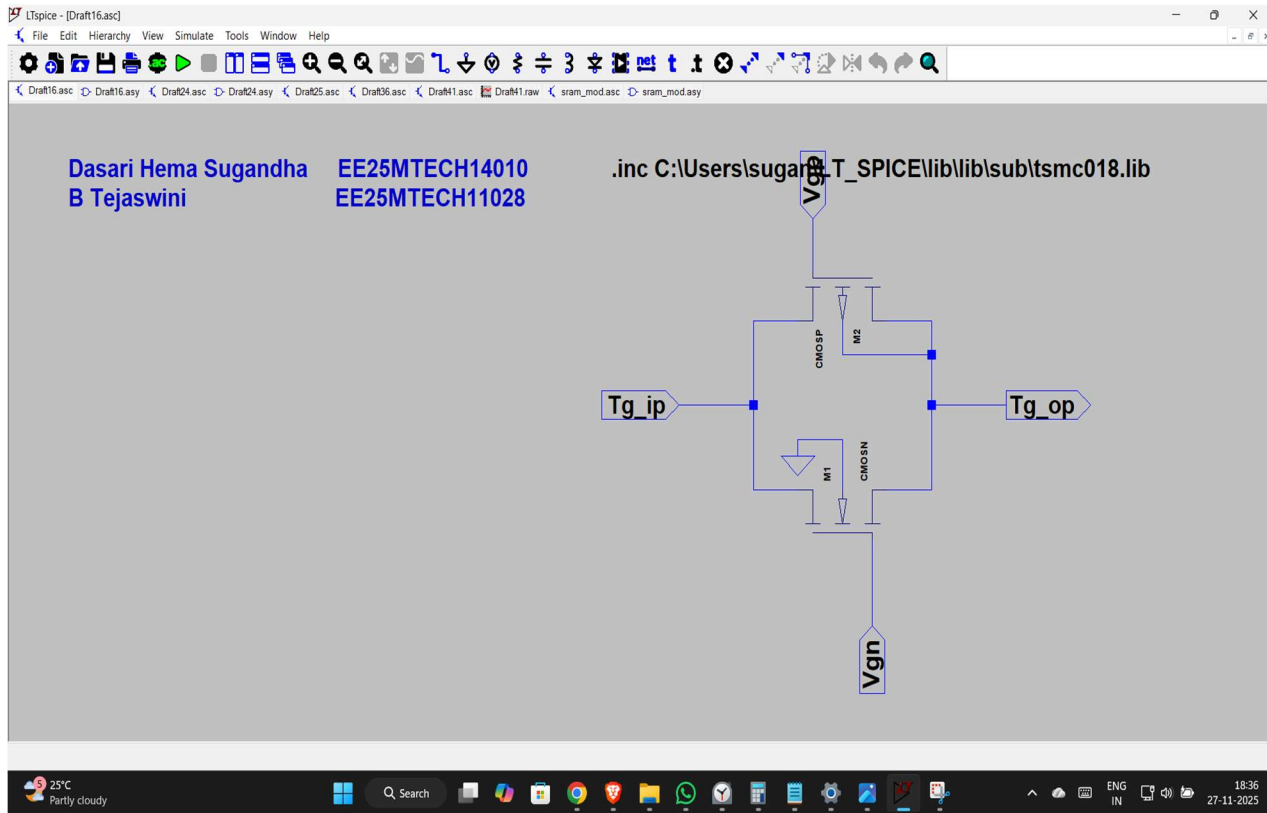
- A SIPO shift register converts a **serial data stream** into **parallel outputs**.
- A data stream is given to the first FF of shift register.
- It is designed such that the data one by one parallely goes to the 1-bit SRAM cell after N cycles(16 clock cycles).
- Allows slow external loading but instant parallel access by the memory.

Schematic:

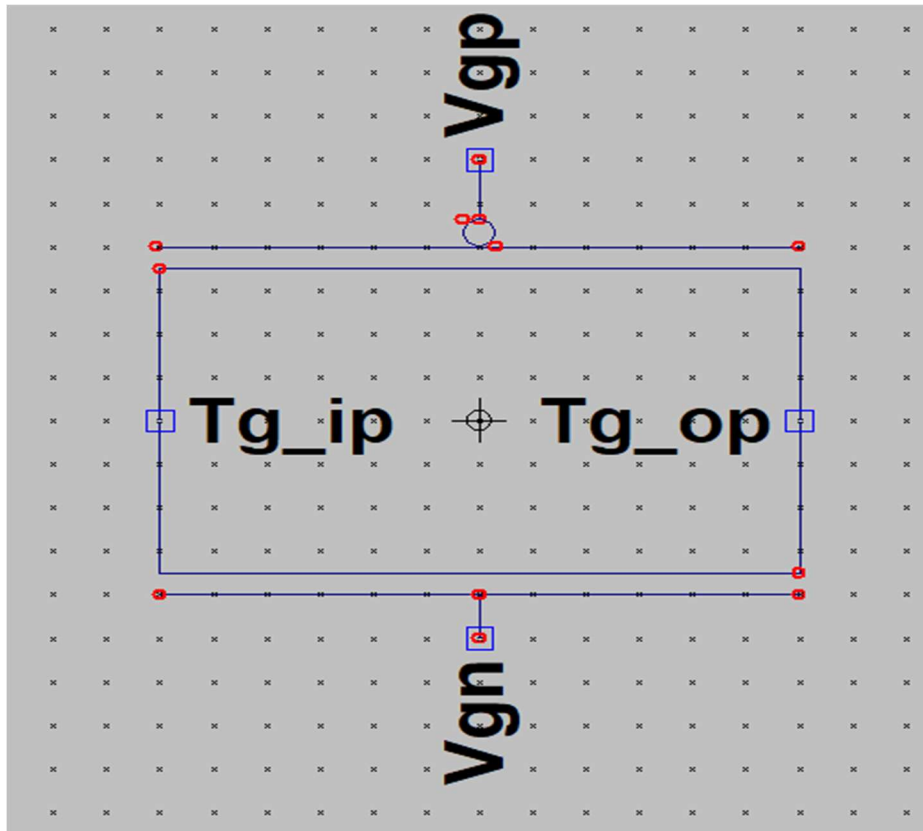


2)Transmission Gate:

- A transmission gate is a **bilateral switch** built from an NMOS and PMOS in parallel.
- It is used in the design to route the data from shift reg to write all the SRAM's.



Symbol:

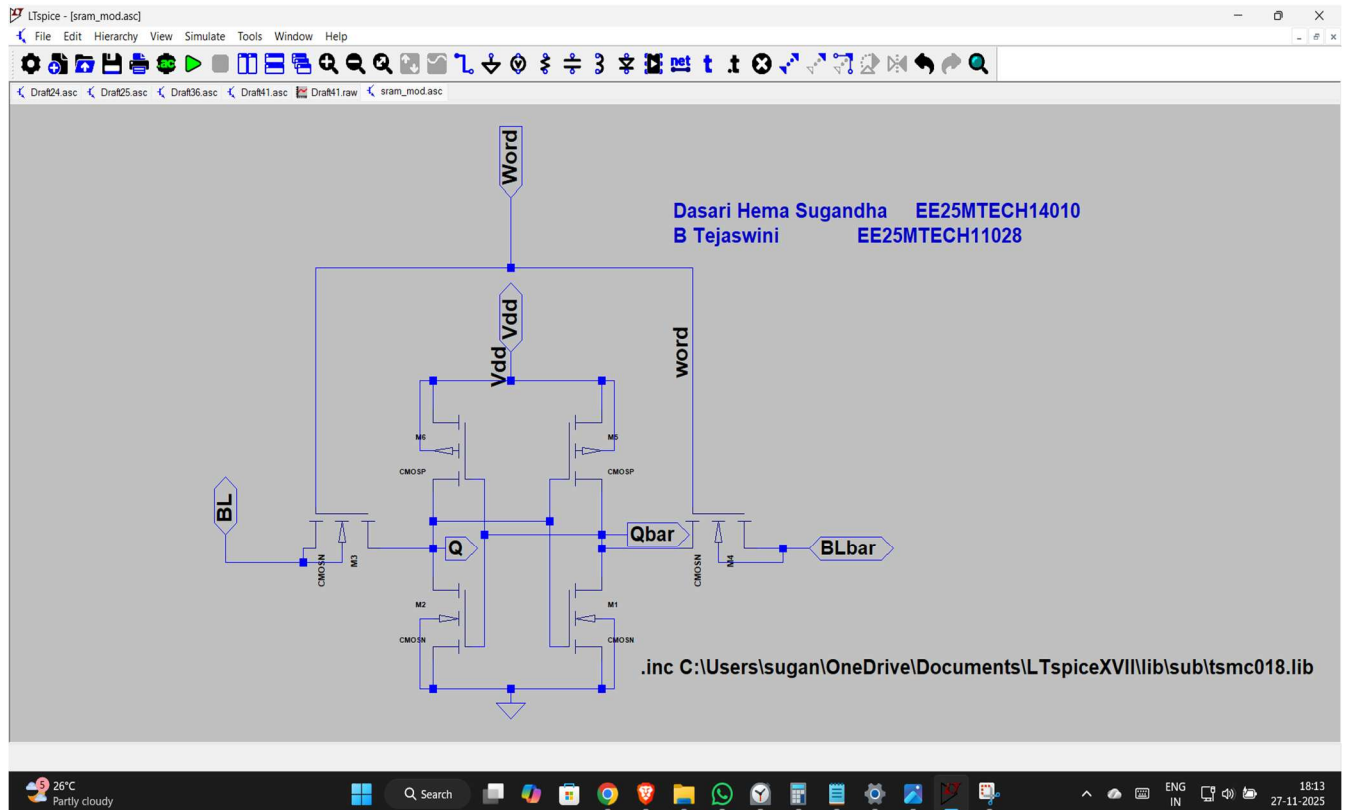


3)SRAM:

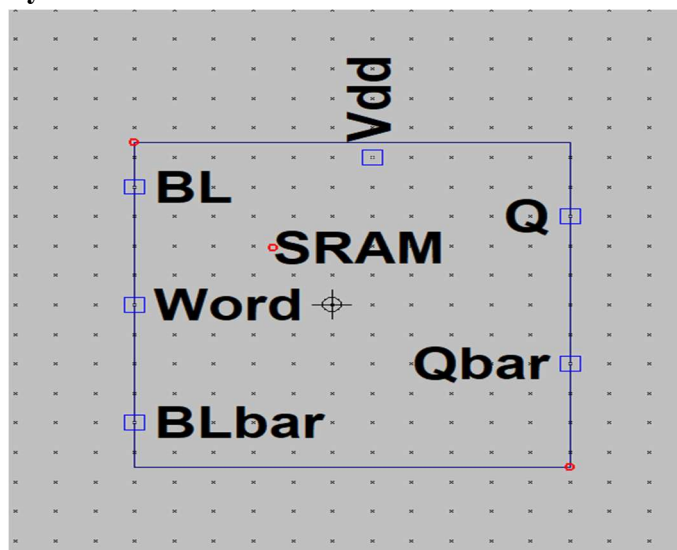
The SRAM bit cell stores one bit using **bistable cross-coupled inverters** and **two access transistors** controlled by a word line.

- Stores the 16 truth-table bits for any 4-input logic function i.e, the shift reg contents are parallely stored into SRAM cells through Transmission gates
- Output stays stable even after power is applied, until rewritten.

Schematic:

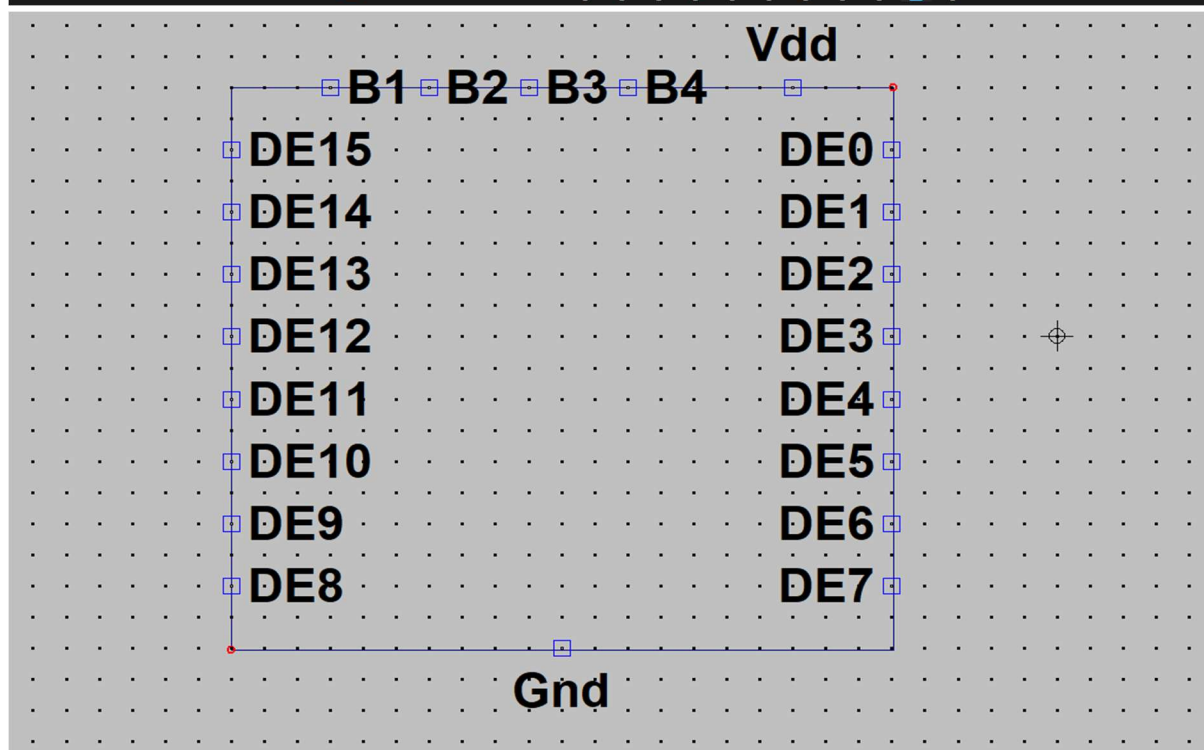
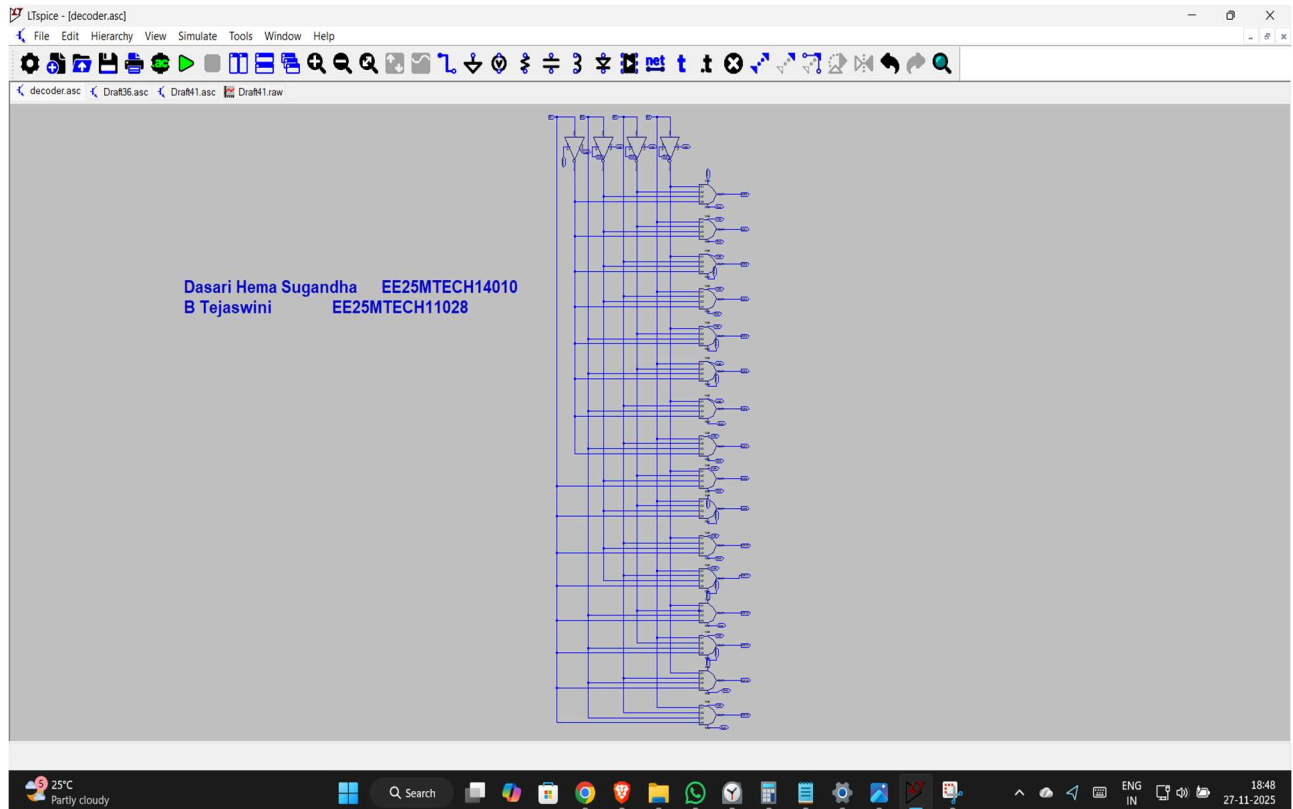


Symbol:



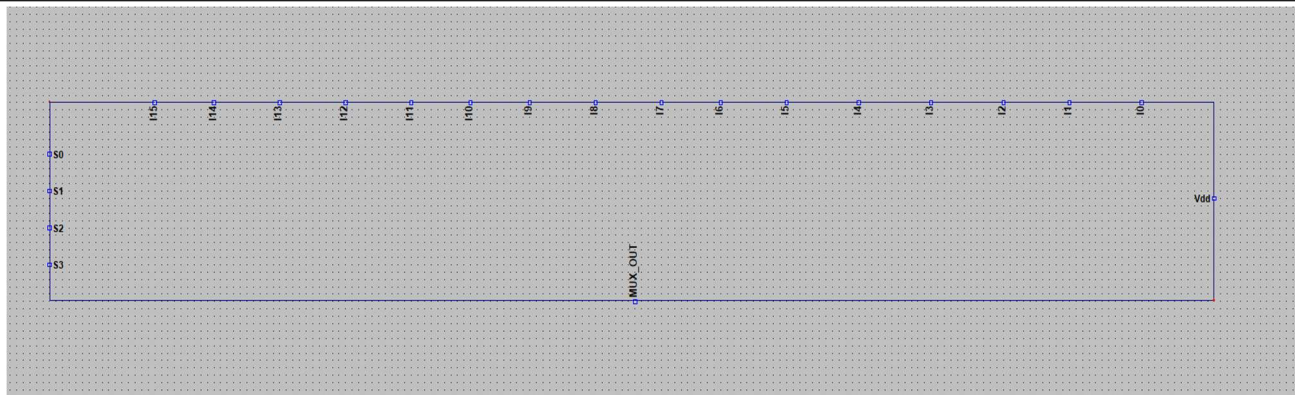
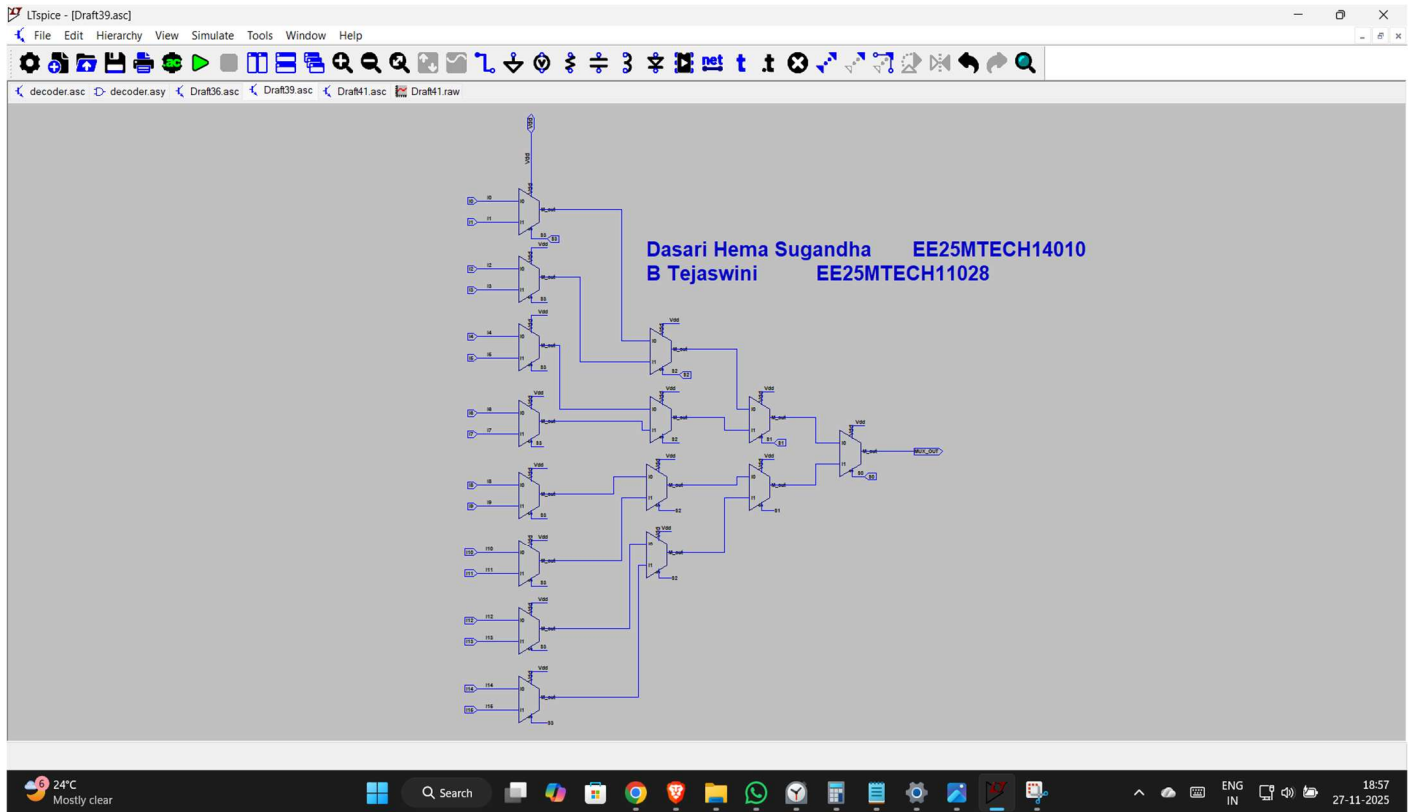
4)DECODER:

- A decoder converts an **n-bit binary value** into **one active output** among 2^n outputs.(one line goes high)
- The decoder 16 bits outputs are given as one input to AND gate and the output from SRAM(Q's) is given as the other input to the AND gate.

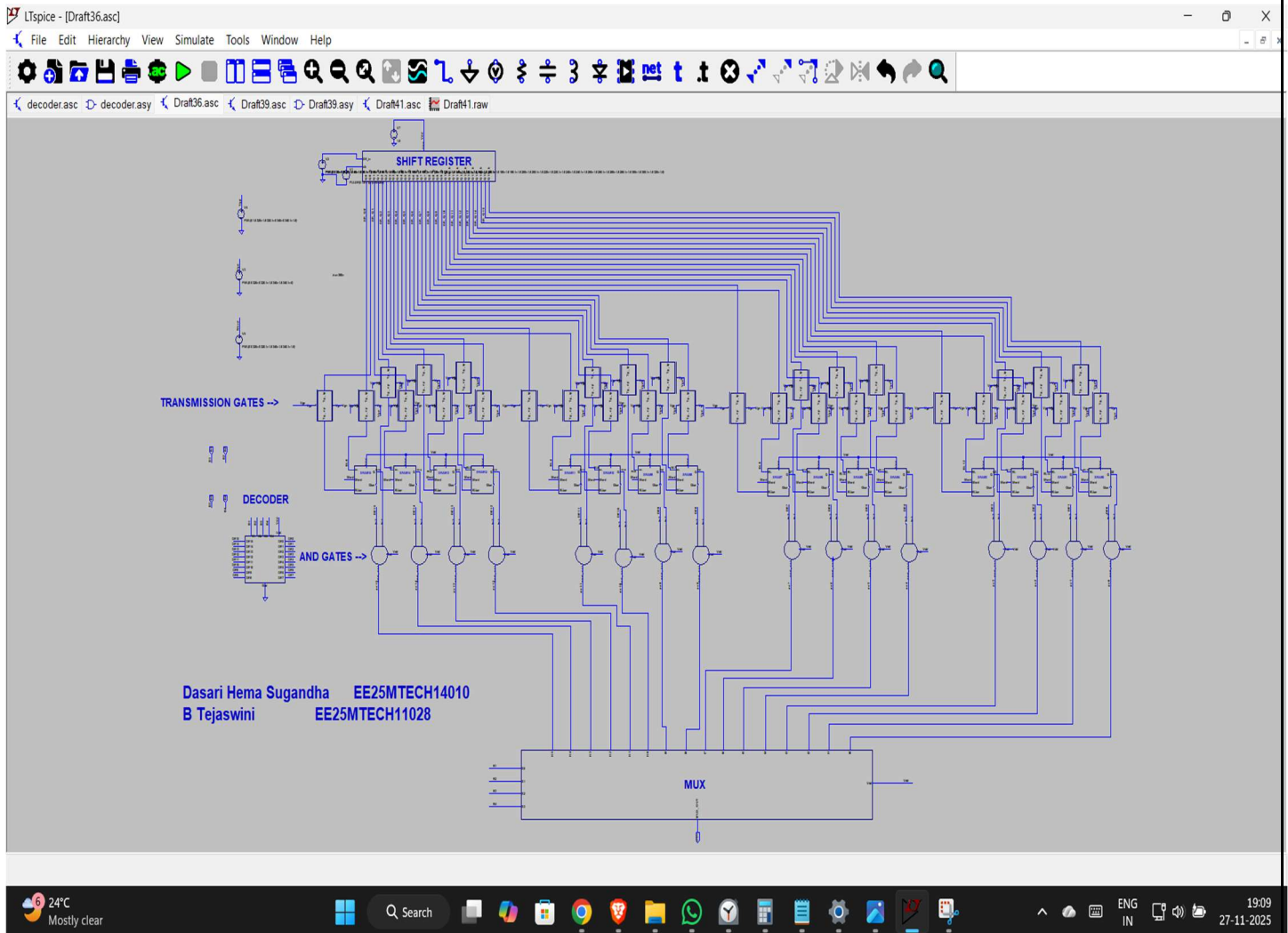


5)MUX:

- In this design, a 16x1 mux is used, which takes the outputs of AND gate as inputs of MUX.
- Here, we designed 16x1 mux using 2x1 mux.
- The select lines to the mux are same as the inputs given to the decoder, so that the same input is selected in the MUX.

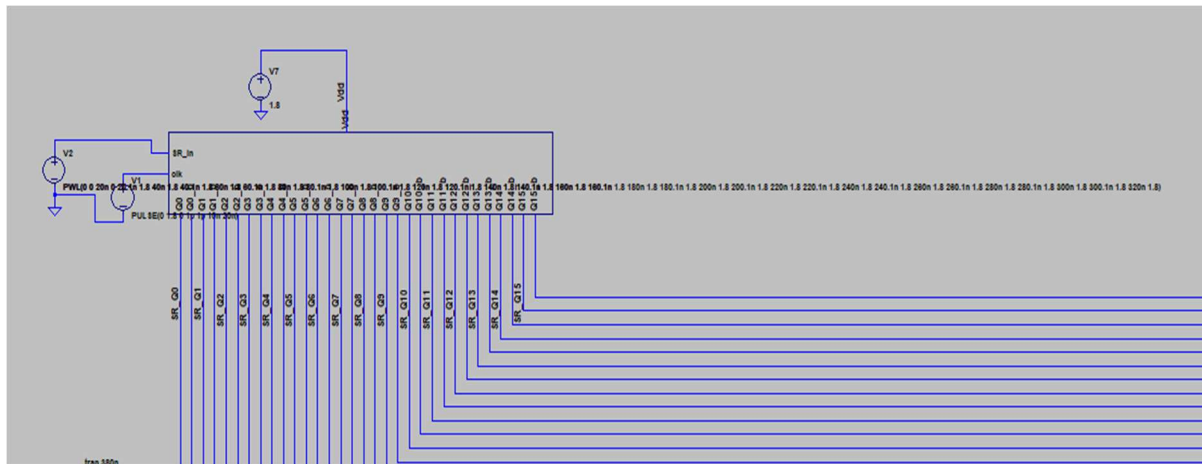


6.SCHEMATIC:

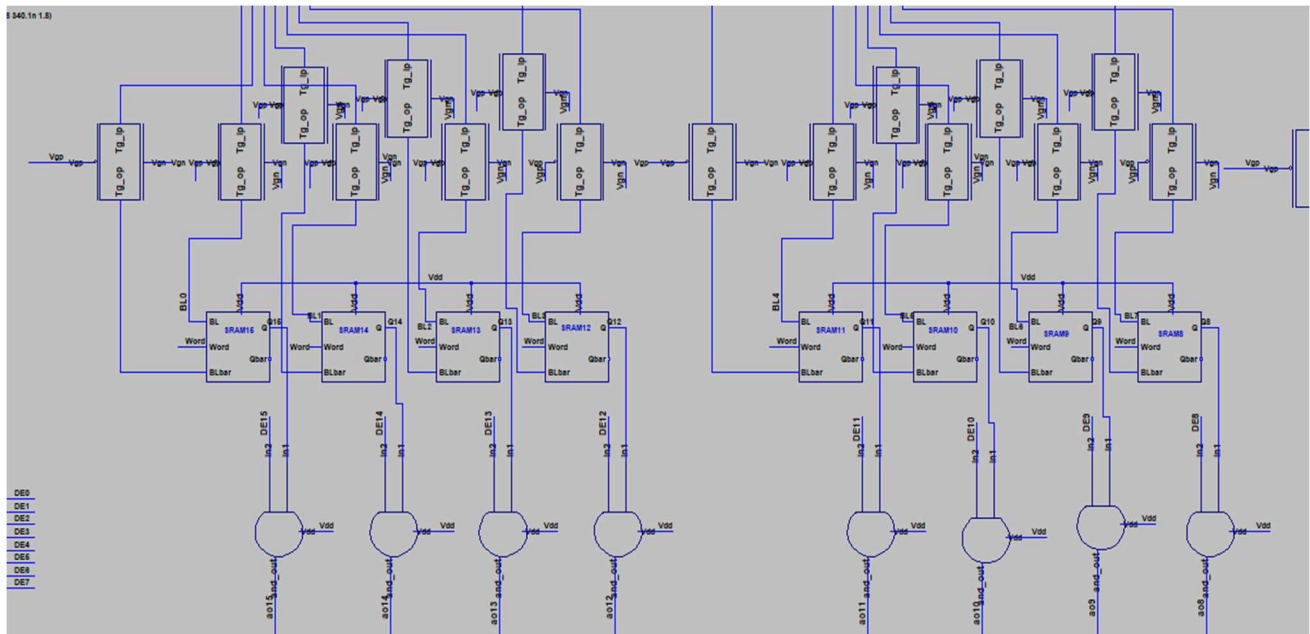


Zoomed pictures:

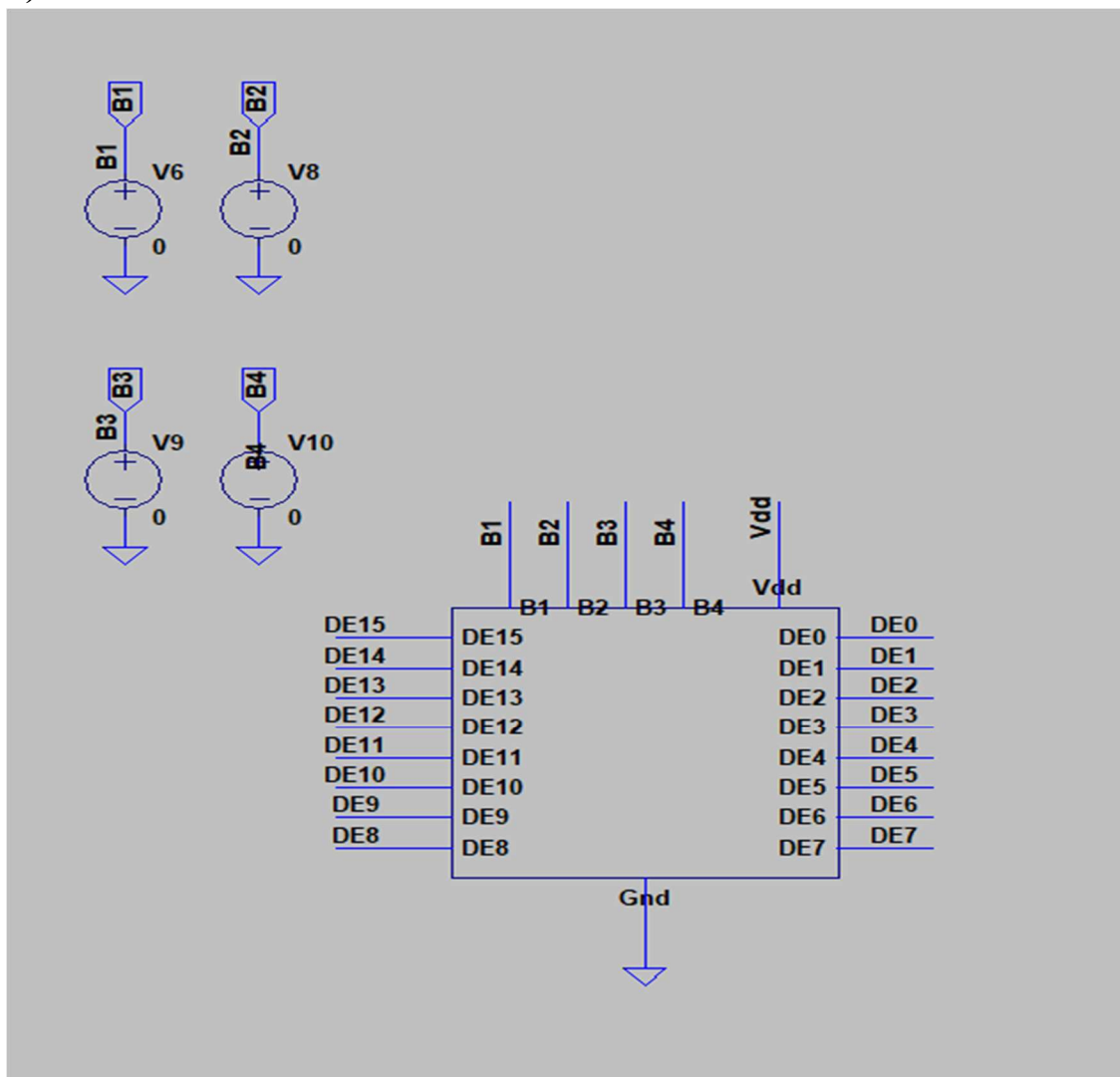
1.Shift register



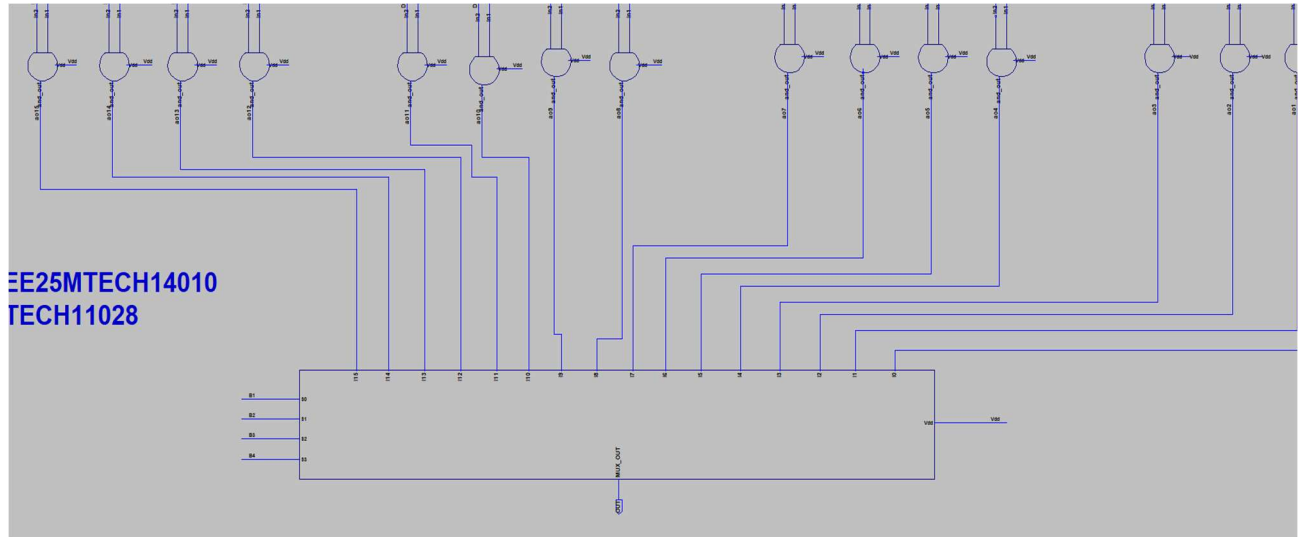
2) Transmission gates:



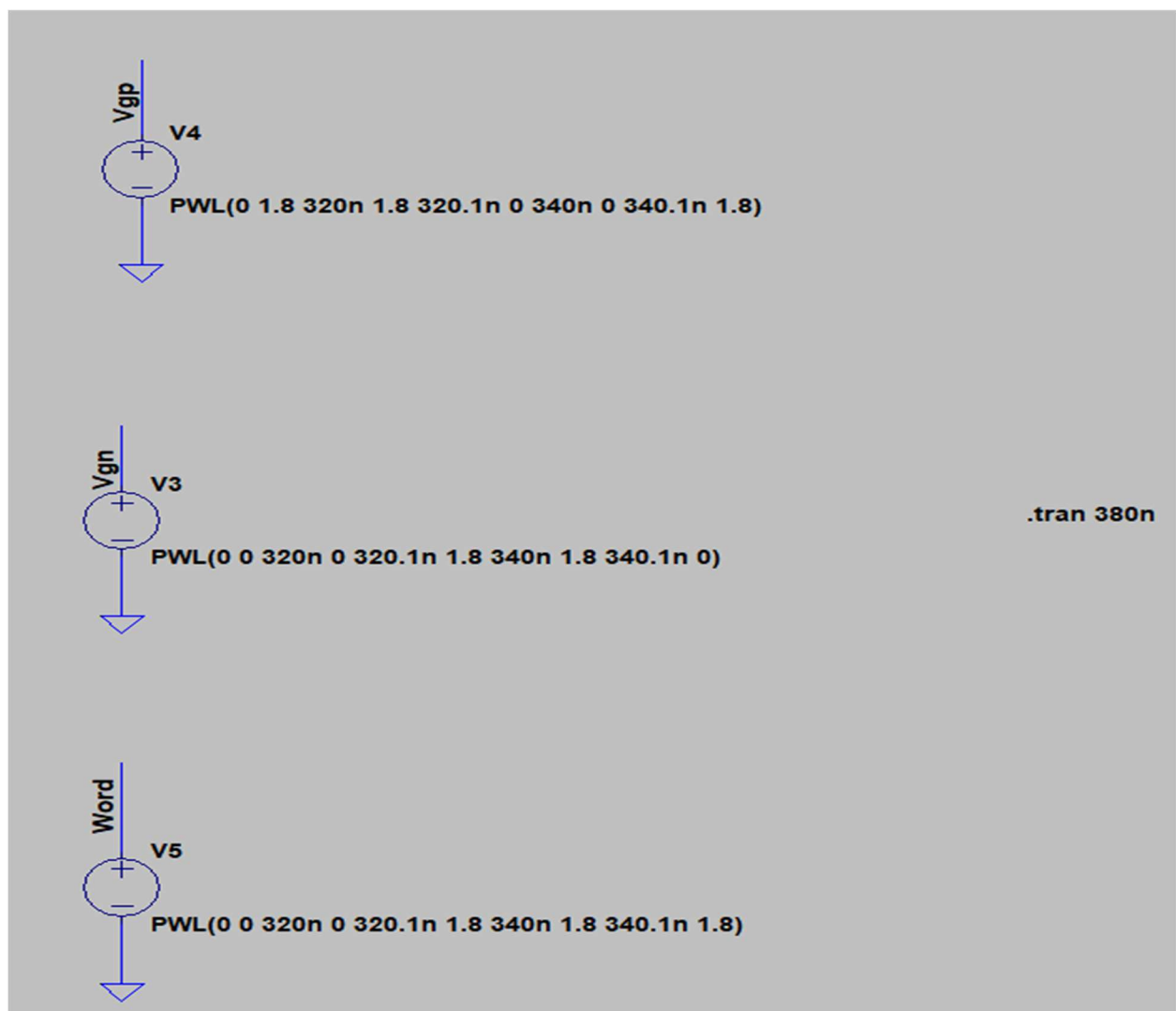
3) Decoder:



4)MULTIPLEXER:



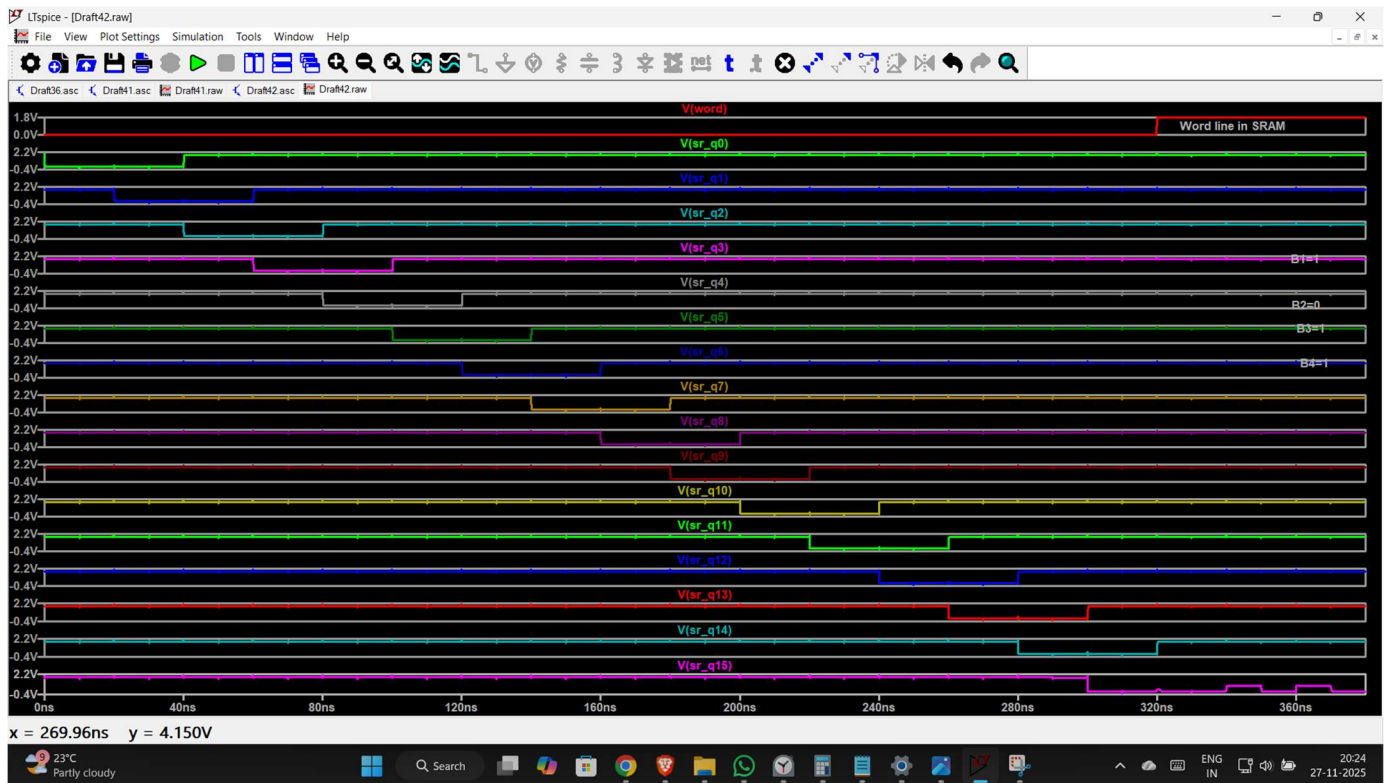
The signals like **VGP** and **VGN** to the transmission gates and **Word** line given to the SRAM are given as below



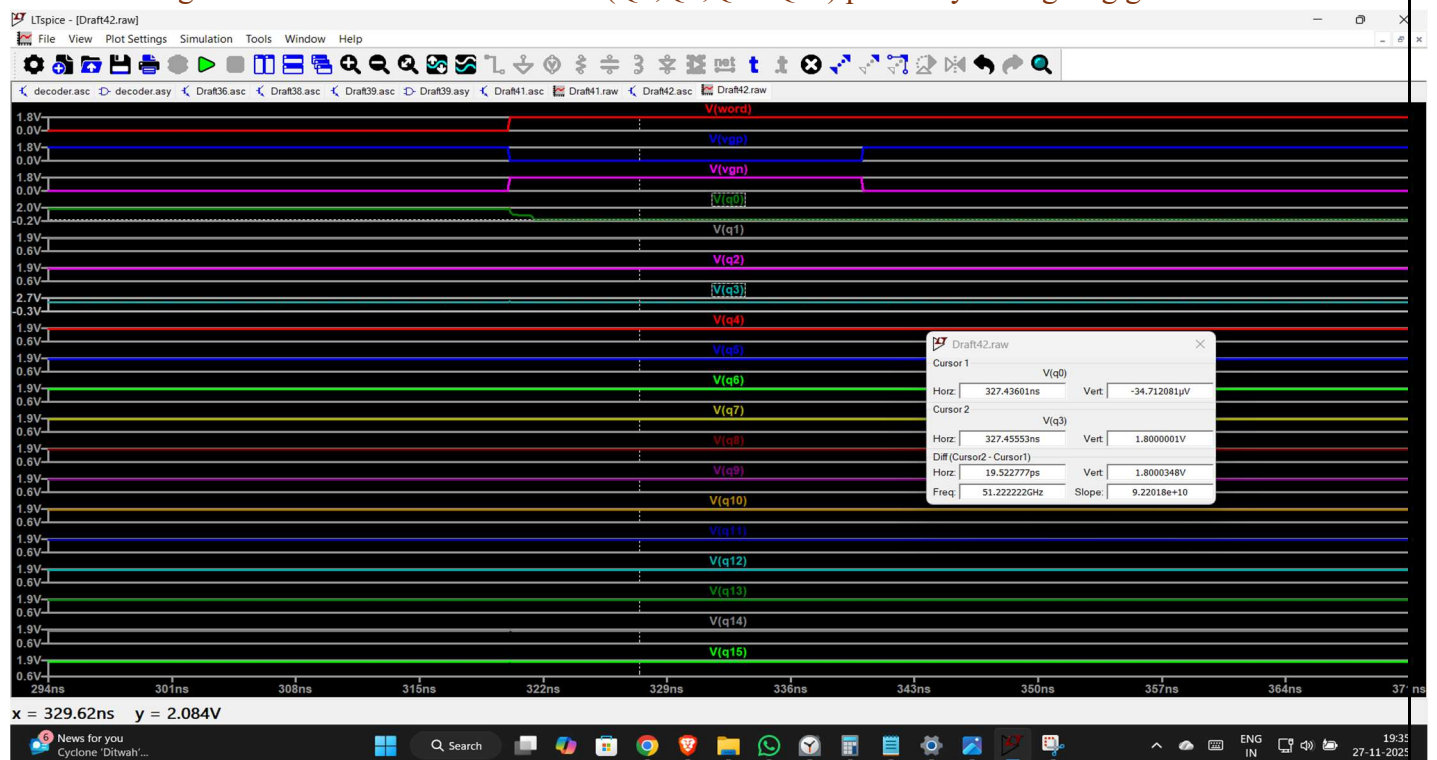
7.SIMULATION RESULTS AND OBSERVATIONS:

Example: Took an OR gate, loaded Shift register with OR gate outputs based on truth table.

- I. **Shift reg outputs taken parallelly** As we can see, after 16 cycles (at 330ns) of clk I got the required bits (view the sr_q15 as the first bit, as the operation of SIPO is this)



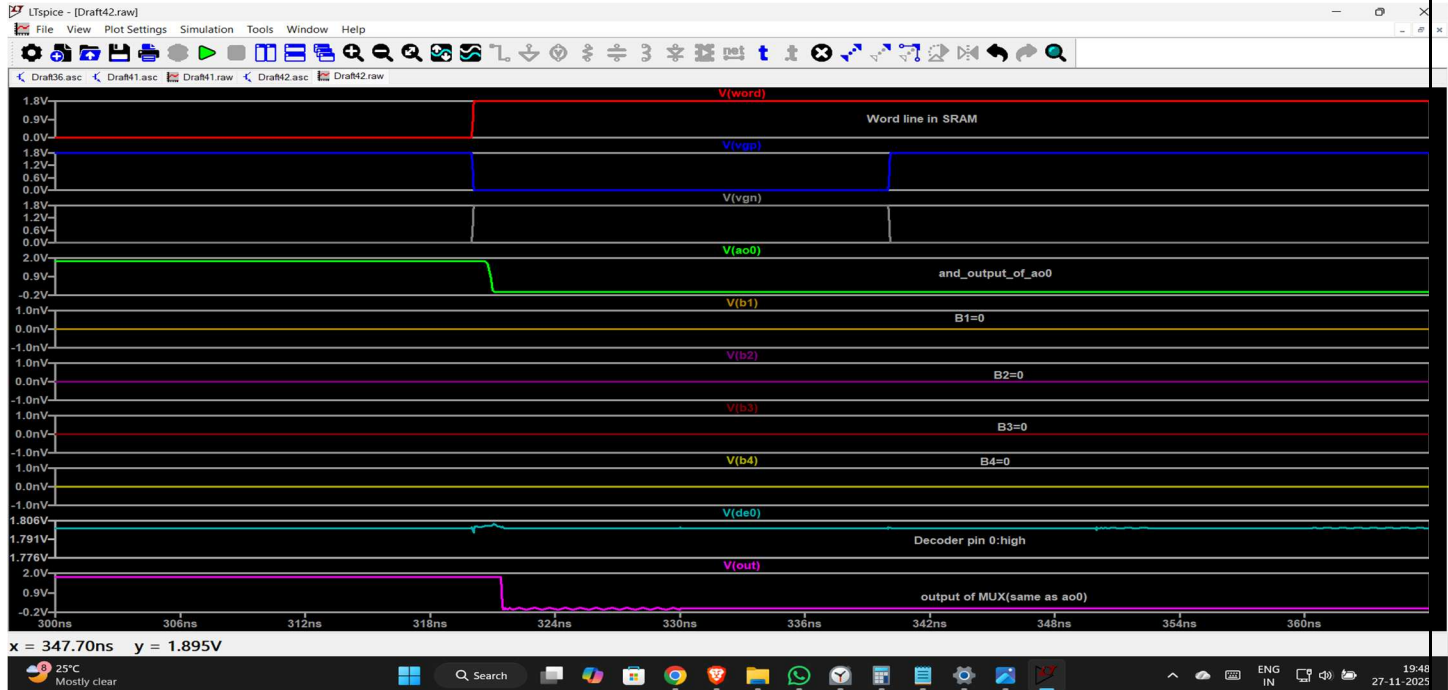
- II. **Shift reg contents are loaded into SRAM(Q0,Q1,Q2...Q15) parallelly through Tg gates**



In the above Result, only $Q0=0$ and all other are logic high '1'(OR GATE)

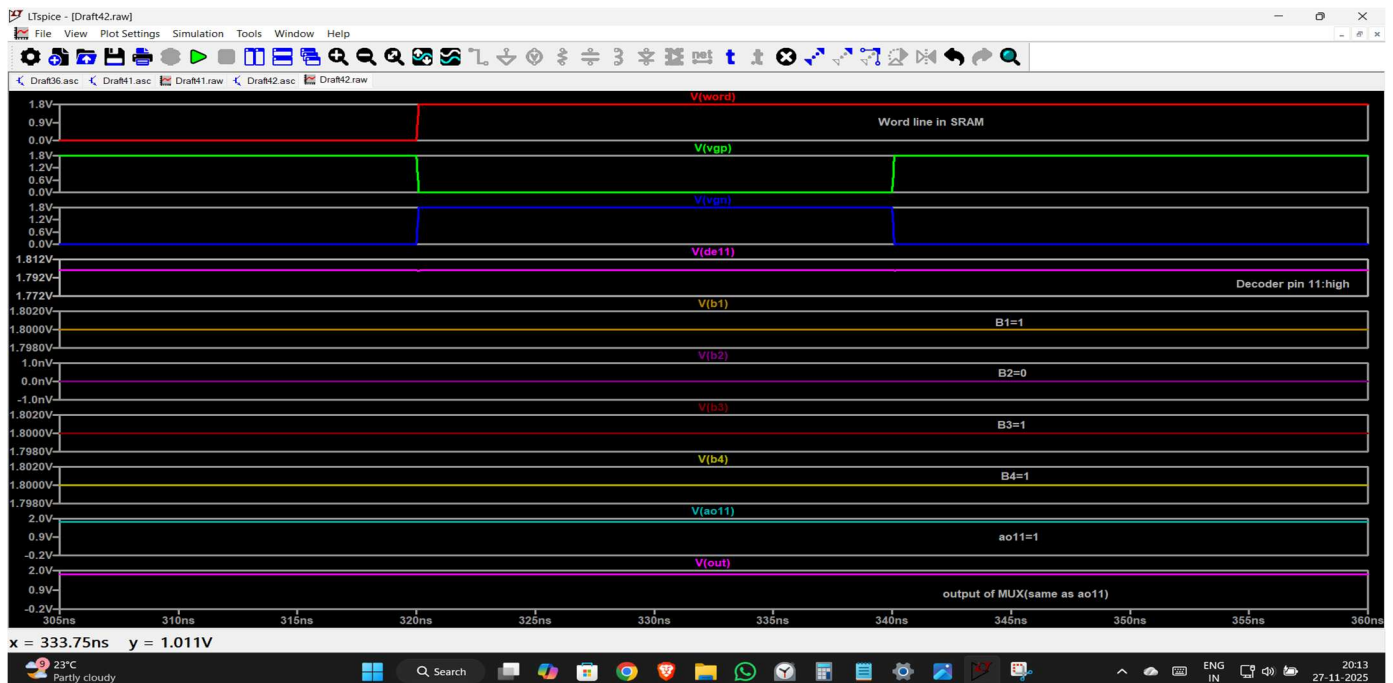
Case:1) $B1=0, B2=0, B3=0, B4=0$ ($B1, B2, B3, B4$ are the inputs to the Decoder)

Then the decoder 'DE0' gets selected (goes High). Now the inputs to the AND gate 'ao0' are DE0 and $Q0$ (output of sram) and the AND gate 'ao0' gets output as '0'. And by $B1,B2,B3,B4$ (also the select lines of mux) the mux also selects the 'I0'.



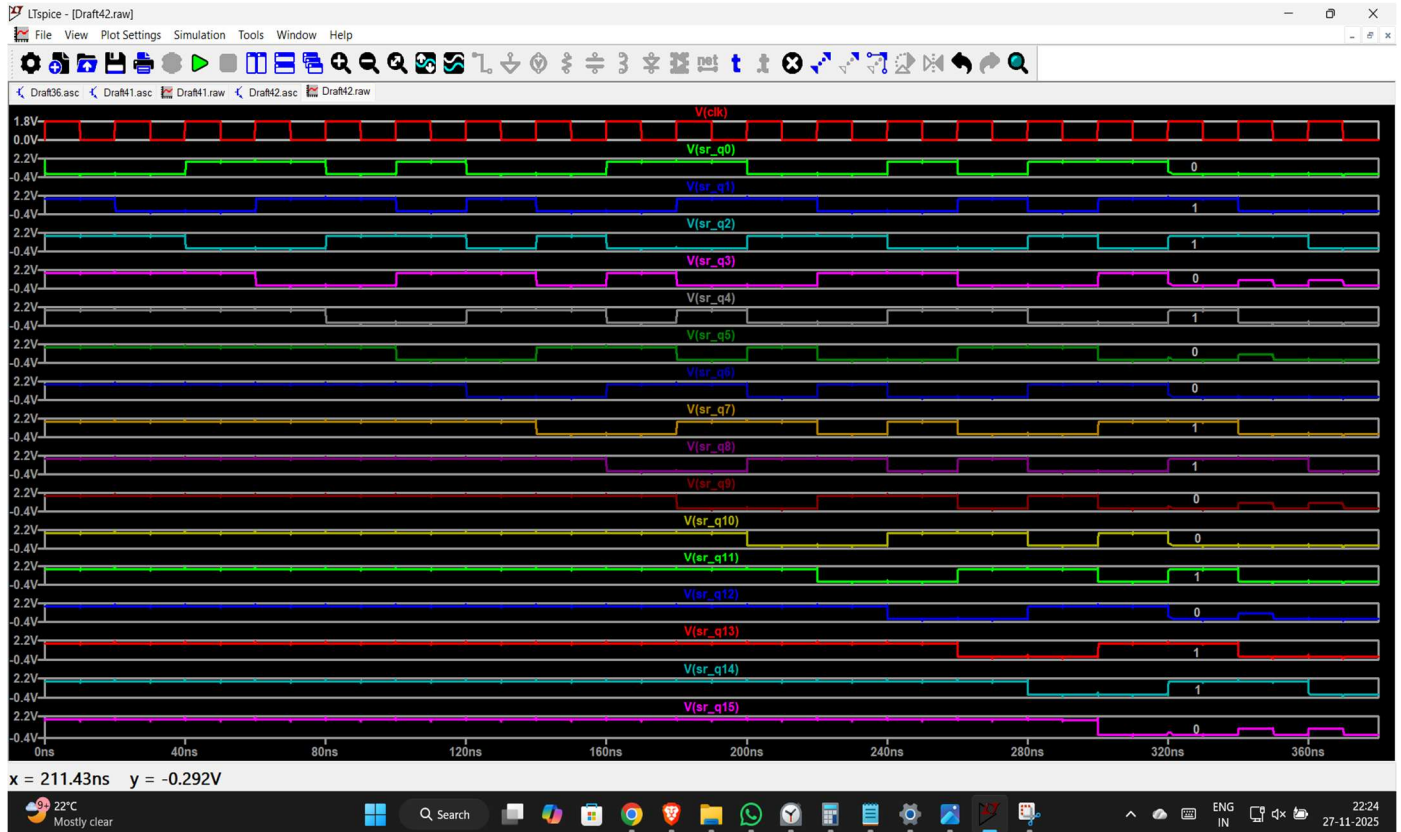
Case:2) $B1=1, B2=0, B3=1, B4=1$ ($B1, B2, B3, B4$ are the inputs to the Decoder)

Then the decoder 'DE11' gets selected (goes High). Now the inputs to the AND gate 'ao11' are DE11 and $Q11$ (output of sram) and the AND gate 'ao11' gets output as '1'. And by $B1,B2,B3,B4$ (also the select lines of mux) the mux also selects the 'I11'.



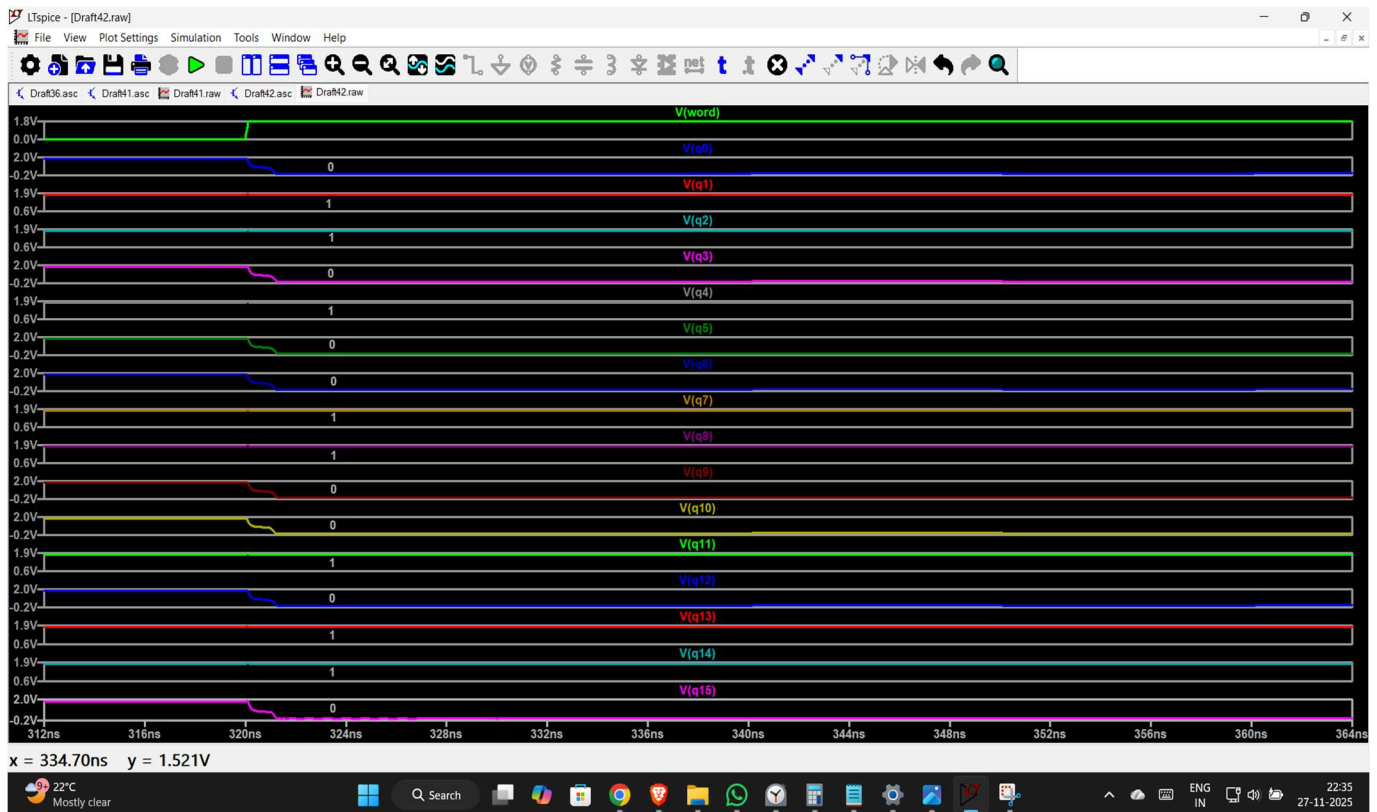
Example: Took an XOR gate, loaded Shift register with XOR gate outputs based on truth table.

I. Shift reg outputs taken parallelly



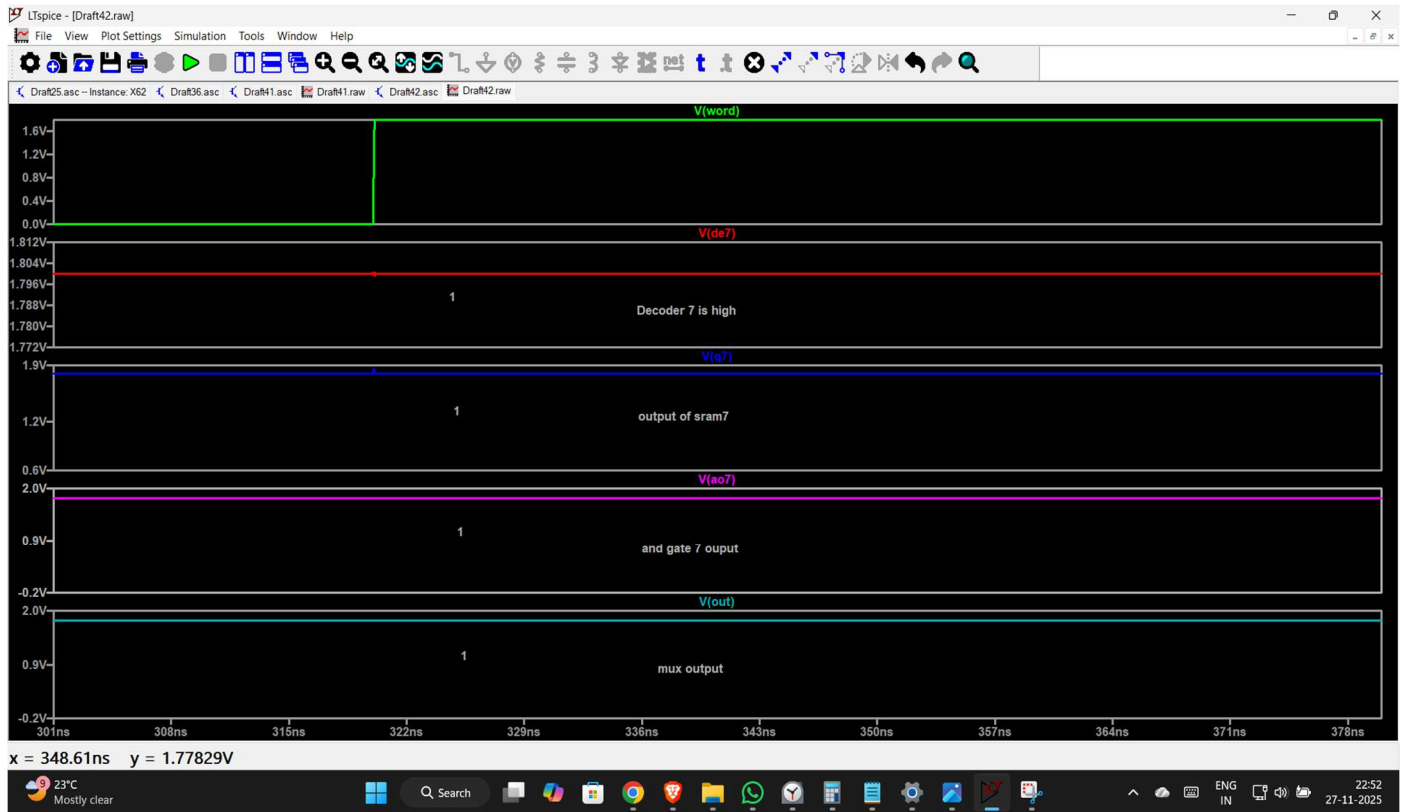
As we can see, after 16 cycles (at 330ns) of clk I got the required bits (view the sr_q15 as the first bit, as the operation of SIPO is this

II. Shift reg contents are loaded into SRAM(Q0,Q1,Q2...Q15) parallelly through Tg gates



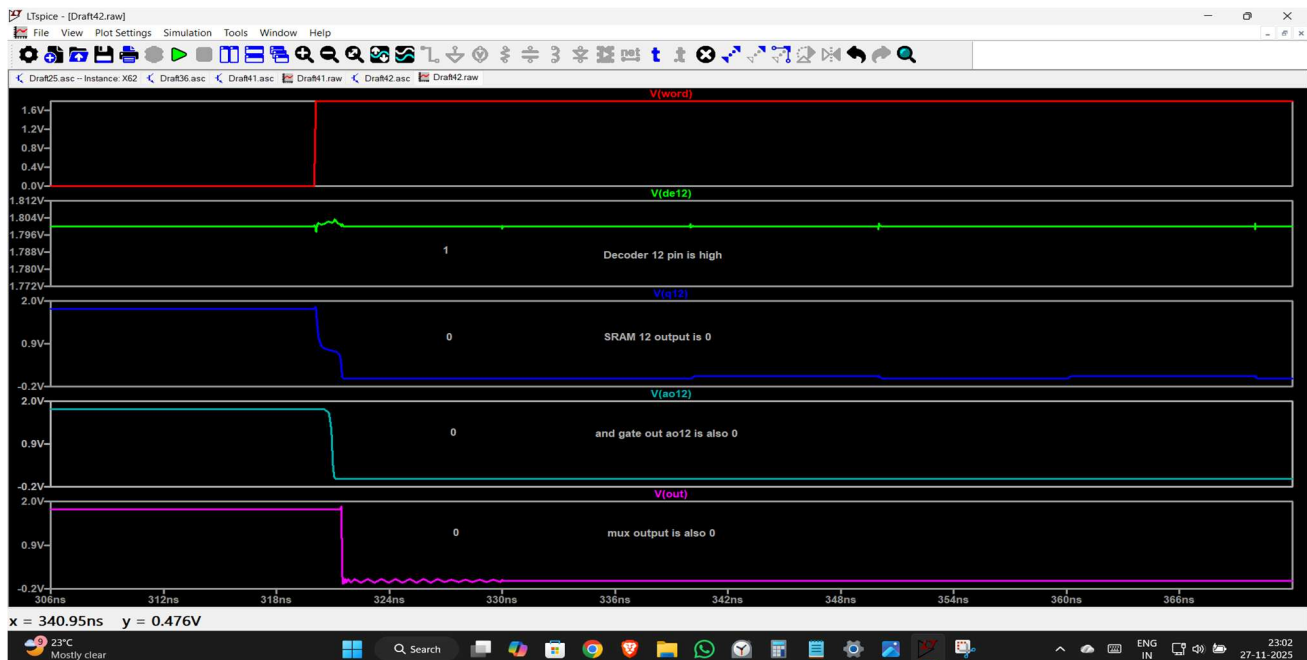
Case:1) B1=0, B2=1, B3=1, B4=1 (B1, B2, B3, B4 are the inputs to the Decoder)

Then the decoder '**DE7**' gets selected (goes High). Now the inputs to the AND gate '**ao7**' are DE7 and Q7(output of sram) and the AND gate '**ao7**' gets output as '**1**'. And by B1,B2,B3,B4(also the select lines of mux) the mux also selects the '**I7**'.



Case:1) B1=1, B2=1, B3=0, B4=0 (B1, B2, B3, B4 are the inputs to the Decoder)

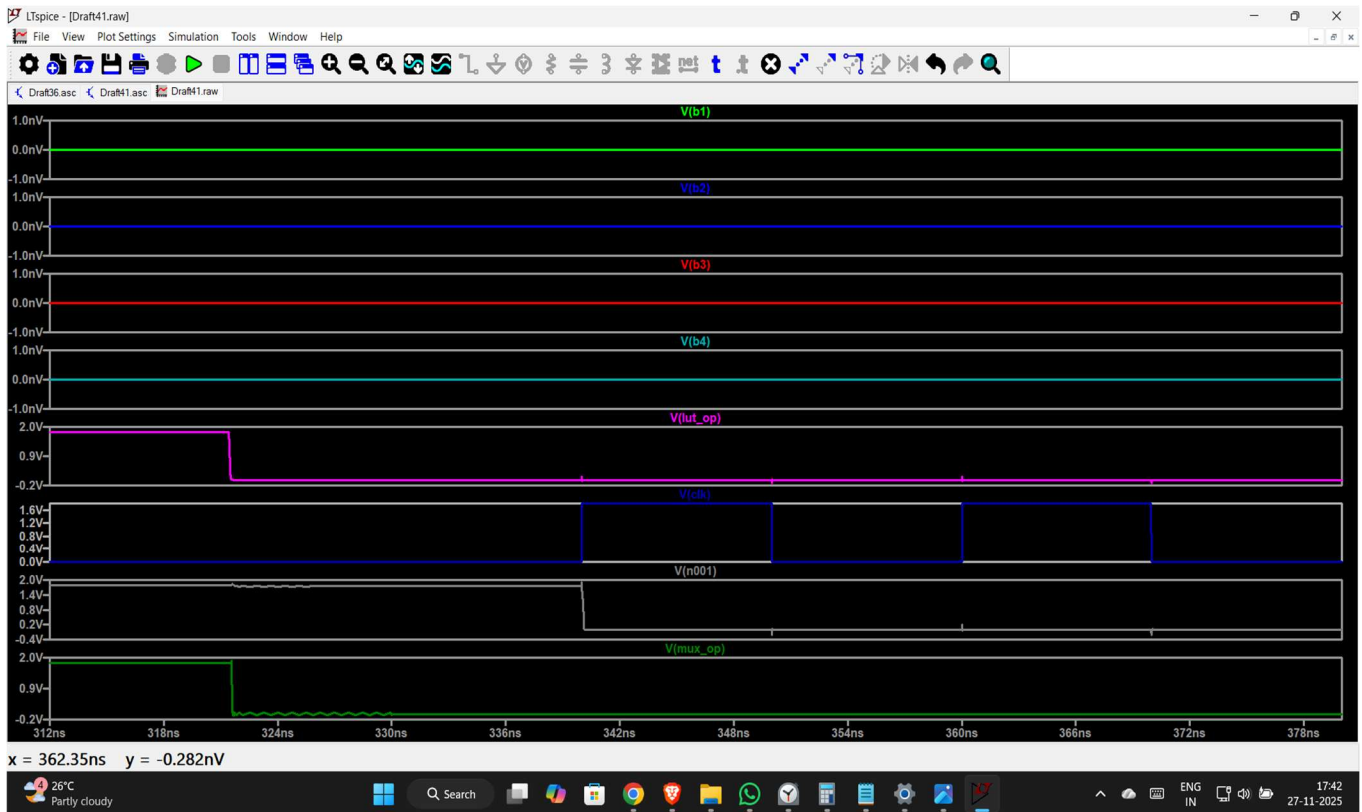
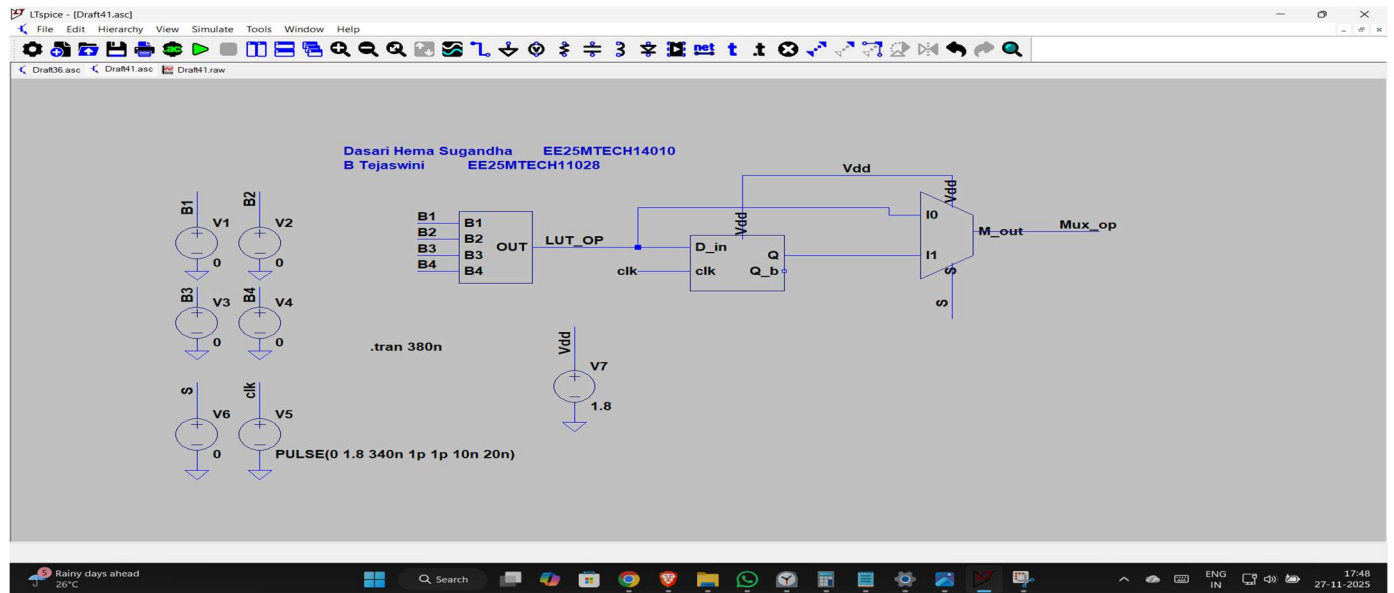
Then the decoder '**DE12**' gets selected (goes High). Now the inputs to the AND gate '**ao12**' are DE12 and Q12(output of sram) and the AND gate '**ao12**' gets output as '**0**'. And by B1,B2,B3,B4(also the select lines of mux) the mux also selects the '**I12**'.



Please refer to the truth table of XOR to check the outputs of Q0,Q1,Q2...Q15.

OPERATION:

1) This is when the select line for 2x1 is 'S=0'. **COMBINATIONAL MODE**

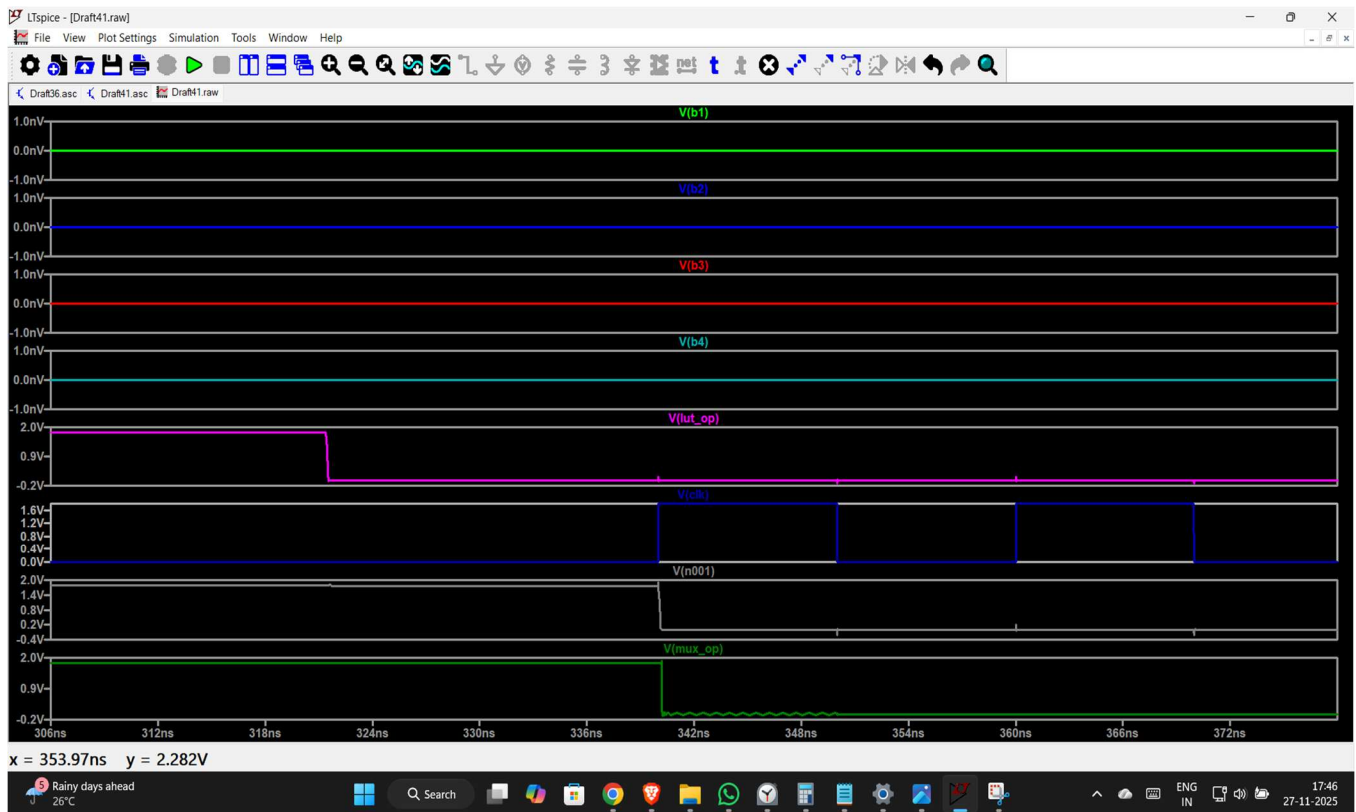
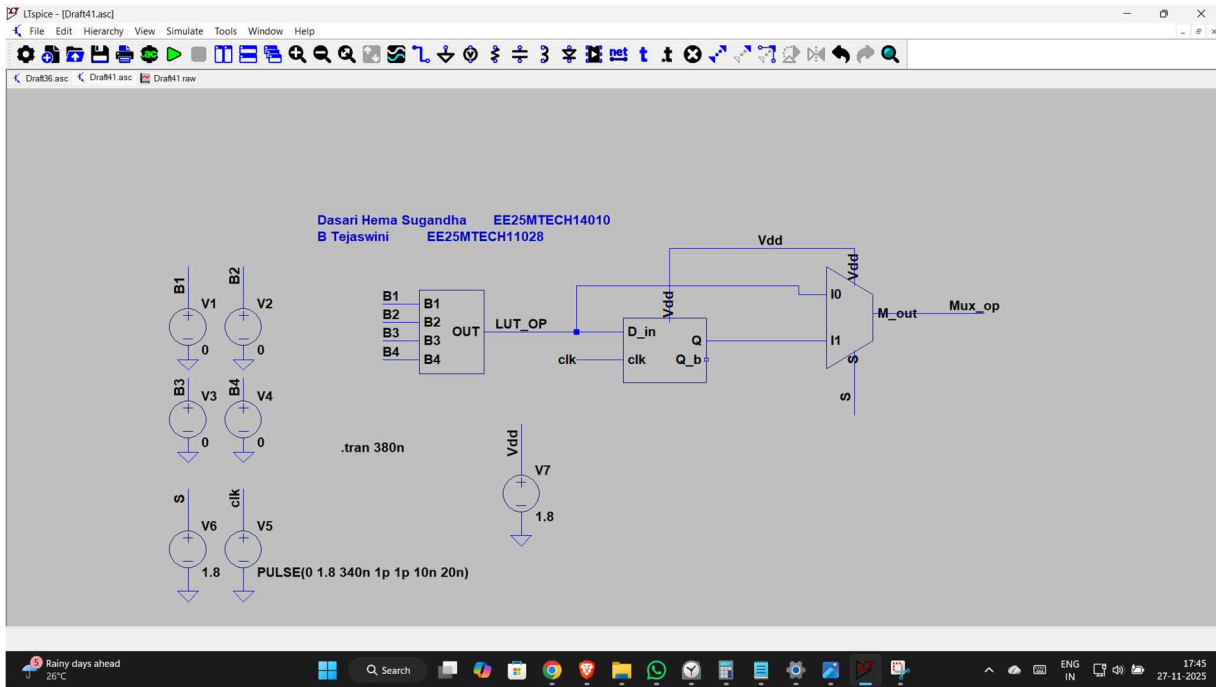


As we can see from the waveform, the V(mux_out) follows the V(lut_op).

Thus the output of LUT is directly routed to the 'Mux_out'

Routing the LUT output directly is nothing but we are using that LUT as the combinational logic i.e., application as LOGIC GATES

2) This is when the select line for 2x1 is 'S=1'. **SEQUENTIAL MODE**



As we can see from the waveform, the $V(\text{mux_out})$ follows the $V(n001)$ instead of $V(\text{lut_op})$ as of previous case [$V(n001)$ is the DFF output which is controlled by $V(\text{clk})$ provided] Here Then the output of LUT is directed to the D_Flip flop and the output of DFF is thus routed to the '**Mux_out**'. By doing so, we are controlling the whole circuitry wrt to clock to further use as synchronous sequential circuits like counters, synchronous circuits, FSM circuits etc.

APPLICATIONS:

- Implementation of arbitrary logic functions using programmable LUTs
- FPGA Configurable Logic Block (CLB) design and study
- Reconfigurable digital logic systems
- Programmable combinational logic (AND, OR, XOR, custom functions)
- Programmable sequential logic using the output D-flip-flop
- Construction of counters, shift registers, and state machines
- Fast digital routing and signal selection using MUX paths

CONCLUSION:

The LUT-based configurable logic block was successfully designed using a SIPO register, SRAM cells, a decoder, a multiplexer, and an optional D-flip-flop. The circuit demonstrates how arbitrary logic functions can be realized purely through memory-based configuration. Decoder-controlled selection and MUX routing enable fast and flexible logic evaluation. The D-flip-flop provides a stable and synchronous output, similar to real FPGA logic blocks. Overall, the design effectively models the core architecture of modern reconfigurable digital systems.